

Install

Typst CLI

Calepin requires Typst 0.15.0 or newer. Install or update the Typst CLI from the Typst installation instructions, and make sure it is available on your PATH.

Calepin

The simplest way to install Calepin is with the official installer script, which works on MacOS and Linux:

```
curl --proto '=https' --tlsv1.2 -LsSf https://github.com/vincentarelbundock/calepin/releases/latest/download/calepin-installer.sh | sh
```

On Windows via powershell:

```
powershell -ExecutionPolicy Bypass -c "irm https://github.com/vincentarelbundock/calepin/releases/latest/download/calepin-installer.ps1 | iex"
```

If you are a cargo for Rust user, you can install with:

```
cargo install calepin
```

Updating Calepin

If you installed Calepin with the official installer, update it with:

```
calepin update
```

This updates only Calepin, using the calepin-update helper installed alongside the main binary. Typst, Python, R, Jupyter, and Jupyter kernels are managed separately.

If calepin update reports that calepin-update is missing, reinstall Calepin with the official installer command above. If you installed Calepin with Cargo, Homebrew, or another package manager, use that tool to upgrade Calepin instead.

Jupyter support

Calepin has built-in support for **Python** and **R**, and can also run many other languages through Jupyter kernels, but that requires installing the language kernel and kernel client tooling.

To use a Jupyter kernel, install `jupyter_client` first:

```
pip install jupyter_client
```

Most kernels then install with a single `pip install`:

```
pip install bash_kernel      # Bash
pip install octave_kernel    # GNU Octave
pip install gnuplot_kernel    # Gnuplot
```

Some Python kernel packages also need to register a Jupyter kernelspec after installation. For Bash, run this in the same Python environment that Calepin uses:

```
python -m bash_kernel.install --sys-prefix
```

If you use `uv run`, the equivalent command is:

```
uv run python -m bash_kernel.install --sys-prefix
```

Some kernels are installed from their language's own package manager:

```
# Julia
julia -e 'using Pkg; Pkg.add("IJulia")'
```

Run `jupyter kernelspec list` to see what engines are registered and available.

Nix

If you use Nix flakes, the default package is the basic Calepin CLI wrapped with Typst on PATH:

```
nix run github:vincentarelbundock/calepin -- --help
nix run github:vincentarelbundock/calepin#calepin -- compile paper.typ
```

To build the package without running it:

```
nix build github:vincentarelbundock/calepin
```

The default development shell is also minimal:

```
nix develop github:vincentarelbundock/calepin
```

For contributor work on the documentation website, use the heavier website shell. It includes Rust tooling, Typst, uv, R packages used by the examples, and the diagram tools used by the website pages.

```
nix develop github:vincentarelbundock/calepin#website
make website
```